Aula 07 · Exercício: encontre o bug

Disciplina: Paradigmas de Linguagens de Programação
Tema: Expressões e Estruturas de Controle (Sebesta, Caps. 7 e 8)
Aluno: Gabriel Santos Furlanette
R.A.: 24000873-2
Curso: Engenharia de Software · Unicesumar

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

Trecho 1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

- **(a) Saída:** `Total: 0012`
- **(b) Regra:** O `for...in` percorre as **chaves** (índices) do array, e elas são strings: `"0"`, `"1"`, `"2"`. Somar número com string faz **concatenação** (coerção implícita): 0 + "0" = "00" → "001" → "0012".
- **(c) Correção:** Usar `for...of`, que percorre os valores: `for (const p of precos)`. Saída: `Total: 60`.

Trecho 2 · Python

```
def desconto(preco, d=None):
    d = d or 10     # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

- **(a) Saída:** `180.0`
- **(b) Regra:** O `or` retorna o primeiro valor **truthy**. Como `0` é falsy, `0 or 10` vale `10`: o desconto de 0% pedido vira 10%.
- **(c) Correção:** Testar explicitamente o `None`: `d = 10 if d is None else d`. Saída: `200.0`.

Trecho 3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
  case 1: nome = "domingo";
  case 2: nome = "segunda";
  case 3: nome = "terça";
  default: nome = "inválido";
}
System.out.println(nome);
```

- **(a) Saída:** `inválido`
- **(b) Regra: Fall-through**: sem `break`, o `switch` entra no `case 2` e continua executando os casos seguintes até o `default`, sobrescrevendo `nome`.
- **(c) Correção:** Colocar `break;` no fim de cada caso, ou usar o switch com seta (Java 14+): `case 2 -> nome = "segunda";`. Saída: `segunda`.

Trecho 4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

- **(a) Saída:** `saldo insuficiente` / `fim`
- **(b) Regra:** O `;` logo após o `if` é um **comando vazio**: ele é o corpo do if. O `printf` seguinte está fora do if e sempre executa. A indentação engana, mas o C a ignora.
- **(c) Correção:** Remover o `;` e usar chaves: `if (saque > saldo) { printf(...); }`. Saída: `fim`.

Trecho 5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

- **(a) Saída:** `dentro: 20` / `fora: 10`
- **(b) Regra: Sombreamento** (shadowing): `:=` declara uma **nova** variável `x` no escopo do bloco do if, escondendo a de fora. A variável externa nunca é alterada.
- **(c) Correção:** Usar atribuição em vez de declaração: `x = x * 2`. Saída: `dentro: 20` / `fora: 20`.

Trecho 6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

- **(a) Saída:** `0.0%`
- **(b) Regra: Divisão inteira**: `7 / 10` entre dois `int` dá `0` (a parte decimal é descartada). Só depois o resultado é convertido para `double`: 0 × 100 = 0.0.
- **(c) Correção:** Converter antes de dividir: `(double) acertos / total * 100`. Saída: `70.0%`.

Resumo

#	Linguagem	Saída	Regra
1	JavaScript	Total: 0012	for...in + concatenação
2	Python	180.0	0 é falsy no or
3	Java	inválido	fall-through sem break
4	C	saldo insuficiente / fim	; vira corpo vazio do if
5	Go	dentro: 20 / fora: 10	sombreamento com :=
6	Java	0.0%	divisão inteira

Conclusão

Nenhum dos trechos dá erro: todos compilam e rodam, mas com resultado errado. São bugs de **semântica** da linguagem — coerção implícita, valores falsy, fall-through, comando vazio, sombreamento de escopo e divisão inteira. Linguagens mais recentes tentam evitar alguns deles (o switch com seta do Java 14, o for...of do JS), mas o programador precisa conhecer as regras. Todos os trechos foram executados para conferir as respostas.